SWIPE TO LEARN

Model Context Protocol (MCP)

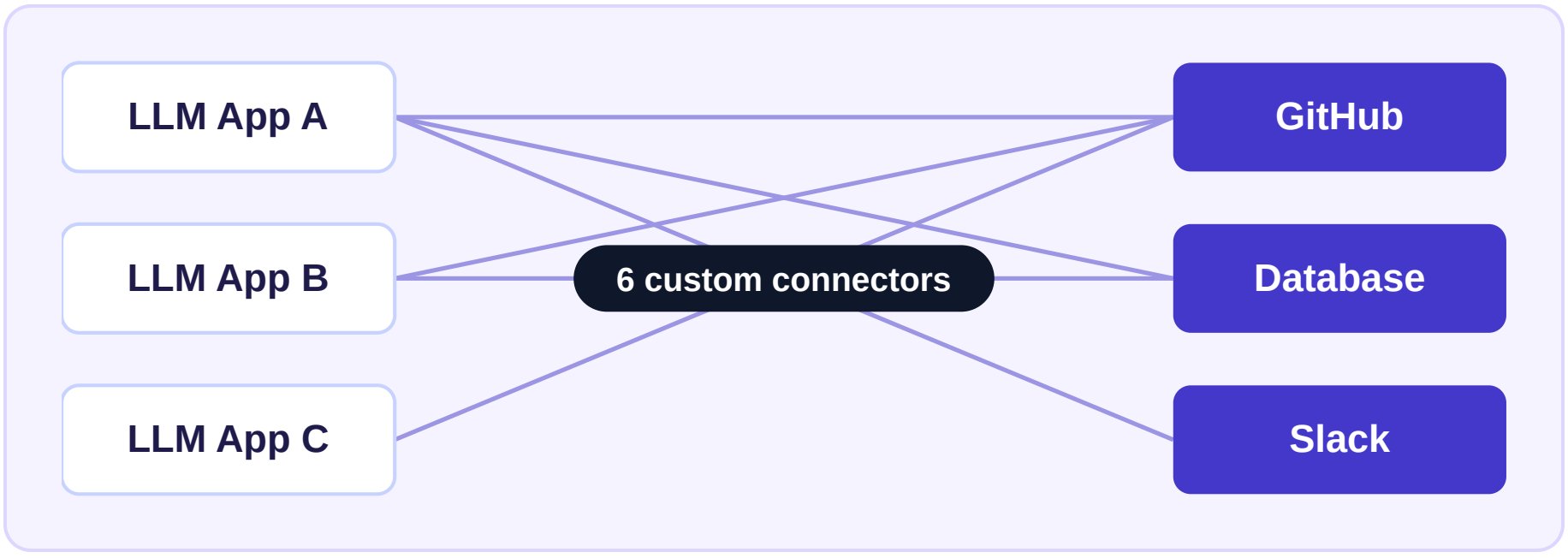
The REST API for AI Agents

How AI applications connect models
to tools, data, and external systems



MCP · AI Agents · Tools · Resources · APIs · Databases

Before MCP: Custom Integrations Everywhere



- Every app **re-implements** each tool's schema
- Auth and permissions are **duplicated** per integration
- Tool discovery is **inconsistent** across apps
- Every new tool or model **multiplies** maintenance

> **More models × more tools = more integration complexity.**

What Exactly Is MCP?

Model Context Protocol (MCP) is an open protocol that standardizes how AI applications connect to external **tools, resources, and prompts**.



Tools
`DO something`

create_github_issue(), query_database(), send_email()

Resources
`READ something`

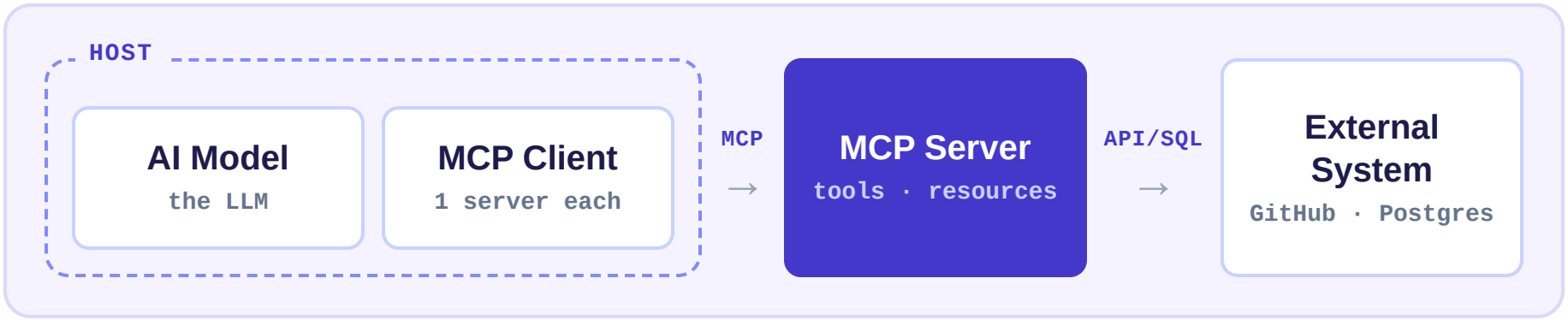
File contents, database schema, docs, config

Prompts
`REUSE a pattern`

Code review or data analysis templates

> MCP standardizes the **connection**. It does not replace your APIs or databases.

Host vs Client vs Server



Component	Responsibility
Host	The AI app or IDE. Runs the model, owns user interaction, and enforces consent and security boundaries.
MCP Client	Lives inside the host. Keeps a connection to one server, sends requests, discovers capabilities.
MCP Server	Exposes tools, resources, and prompts. Talks to external systems.
External System	The real database, API, filesystem, or cloud service.

> The MCP server is **not** the database. It is the protocol-facing layer in front of it.

Messages & Transports: How MCP Talks

CLIENT → SERVER

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list"
}
```

SERVER → CLIENT

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": { "tools": [...] }
}
```

MCP messages are **JSON-RPC 2.0**: a `method`, optional params, and an `id` that ties each response to its request. It is a protocol, not a REST API.

Local: stdio

`host → subprocess`

The host launches the server as a child process and exchanges messages over stdin/stdout. Great for local dev tools.

Remote: Streamable HTTP

`host → HTTP endpoint`

The server runs independently and serves clients over HTTP, optionally streaming with SSE. Replaced the older HTTP+SSE transport.

> **JSON-RPC** defines the message. The **transport** carries it.

What Happens When an Agent Calls a Tool?



> **On connect:** `initialize` → `tools/list`. The model already knows `list_issues`.

1 **User asks**
"Find my latest GitHub issues."

2 **Model picks a tool**
It needs GitHub data, so it selects `list_issues` from the discovered tools.

3 **Client sends `tools/call`**
Structured arguments go to the server. The host may ask the user to approve first.

4 **Server validates, then calls GitHub**
It checks arguments and permissions, then calls the GitHub API.

5 **Structured result flows back**
GitHub → MCP server → MCP client → model.

6 **Model writes the answer**
It reads the result and responds to the user.

A Simple MCP Server in Python

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("orders")

@mcp.tool()
def get_order(order_id: str) -> dict:
    """Look up an order by ID."""
    return {"order_id": order_id,
            "status": "shipped"}

if __name__ == "__main__":
    mcp.run()  # stdio by default
```

Type hints
Become the tool's input schema

Docstring
Becomes the description the model reads

tools/list
Clients discover `get_order` automatically

Same pattern for a database server: `list_tables`, `describe_table`, `run_readonly_query`. Never unrestricted SQL.

SDK APIs evolve. Verify the current docs before shipping production code.

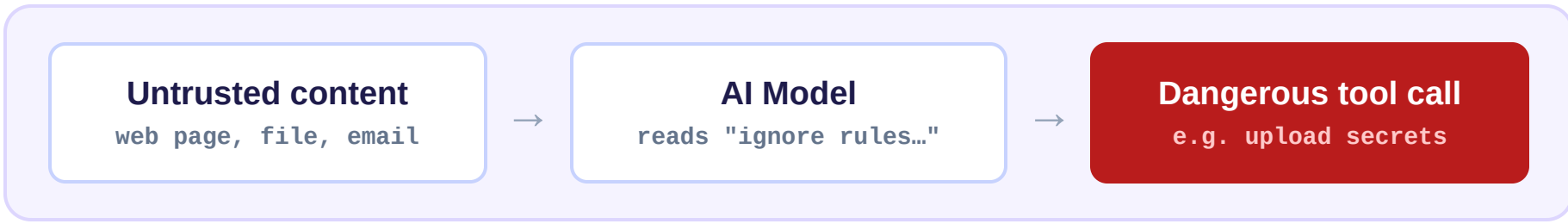
MCP vs REST: Different Jobs

REST API	MCP
Application-to-application communication	AI application-to-capability communication
Developer-defined endpoints	Capabilities a client can discover at runtime
Consumers are usually known ahead of time	Built for AI clients that discover tools
Contract: endpoints and schemas	Contract: tools, resources, prompts



> MCP sits **above** your existing APIs. "REST for AI agents" is an analogy, not a replacement.

Tool Access Is Capability Access



Prompt injection: hidden instructions in content can push a model to misuse its tools.

Secure it

- Authenticate and authorize every call
- Least privilege; split read / write
- Validate every tool argument
- Keep secrets away from the model
- Human approval for risky actions

Operate it

- Timeouts, retries, graceful failures
- Audit log for every tool call
- Track latency and error rates
- Rate limits and network controls
- Version tools and assign owners

> MCP does not make tool use safe by itself. Treat an MCP server like a **production backend service**.

MCP Is a Bridge, Not the Backend

A standardized bridge between AI applications and the capabilities your backend already provides.

- MCP standardizes how AI apps reach tools, data, and prompts
- Clients connect hosts to servers; servers expose capabilities
- JSON-RPC defines messages; stdio or HTTP carries them
- It works alongside your REST APIs and databases
- Tool access is capability access: secure it like production



♥ Like this post

↻ Repost it

\+ Follow for more